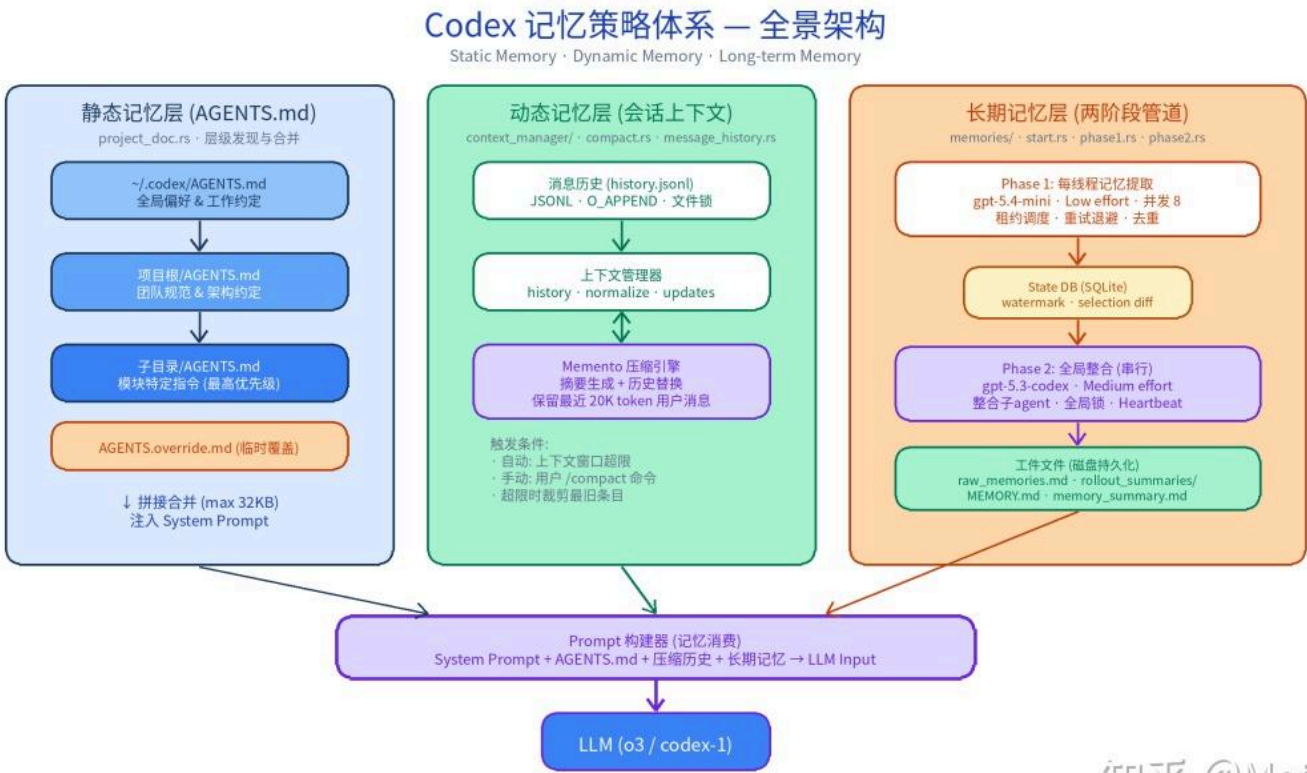


Codex 的记忆系统

<https://zhuanlan.zhihu.com/p/2028586838806344324>

全生命周期总览

在深入各章之前，先建立整体认知。**Codex 的记忆系统并非各模块的简单堆叠，而是围绕 产生 → 存储 → 使用 → 维护 这条数据流紧密协作的有机整体。**



知乎 @Meta

整个系统已从 TypeScript 完全重写为 Rust，核心代码位于 `codex-rs/core/src/` 目录下。下表勾勒出生命周期各阶段的关键模块：

生命周期阶段	核心问题	关键模块	章节
什么值得记住	哪些信息应进入记忆系统？	contextual_user_message.rs	第一章
记忆如何产生	谁来提取、何时触发？	memories/ start.rs、Phase 1/ Phase 2 管道	第二章
记忆存在哪里	静态文件还是数据库？ 作用域 如何隔离？	project_doc.rs、 memories/ storage.rs、 State DB	第三章
记忆如何使用	如何组装进 Prompt？如何过滤？	Prompt 构建器 、片段分类	第四章
记忆如何维护	如何遗忘过时信息？如何增量更新？	Selection Diff、 Watermark 、文件清理	第五章

概念辨析：[存储层](#) vs 生产者

阅读本文时，请始终区分谁在产生记忆和记忆存在哪里。以两阶段管道为例：Phase 1 和 Phase 2 是生产者——它们执行提取和整合；`raw_memories.md` 和 State DB 是存储层——它们承载结果。两者通过 `storage.rs` 连接，将生产者的输出[持久化](#)到存储层。

本文按上述五个阶段展开，最后提炼设计模式与启示。

一、什么值得记住——记忆的定义与边界

在展开「如何产生」之前，必须先回答一个更根本的问题：[什么信息值得成为记忆？Codex 对此给出了清晰的答案——通过片段分类机制划定记忆的边界。](#)

1.1 片段分类：记忆的入口过滤器

`contextual_user_message.rs` 实现了关键的记忆排除过滤。不是所有上下文片段都应该进入记忆管道：

代码块

```
1 pub(crate) fn is_memory_excluded_contextual_user_fragment(content_item:
  &ContentItem) -> bool {
2     let ContentItem::InputText { text } = content_item else {
3         return false;
```

```
4     };
5     // 排除：AGENTS.md 指令和 Skill 负载（已在 Prompt 中或临时性内容）
6     // 保留：环境上下文、用户命令、子 Agent 通知、中断通知等
7     AGENTS_MD_FRAGMENT.matches_text(text) || SKILL_FRAGMENT.matches_text(text)
8 }
```

这个分类体系的设计逻辑是排除可推导和已注入的内容，保留不可推导的隐式知识：

片段类型	是否排除	理由
AgentsMd	✔ 排除	已通过静态记忆层注入 System Prompt，再进入记忆管道会导致重复
SkillPayload	✔ 排除	工具调用是临时性的，不应被记忆
EnvironmentContext	✘ 保留	环境信息（如 OS、shell 类型）对跨会话理解有价值
UserShellCommand	✘ 保留	用户的 命令模式 反映工作习惯
SubagentNotification	✘ 保留	子 Agent 的结果可能包含有价值的上下文
TurnAborted	✘ 保留	中断原因可能暗示用户偏好

1.2 设计哲学：不保存可推导信息

从源码的过滤逻辑中，可以提炼出一条核心原则：**记忆系统只应保存不可推导的信息。**

代码模式、架构、文件路径等可以通过 `grep`、`git log` 等工具实时获取的内容，不应进入记忆管道。AGENTS.md 中已有的规则同样不需要重复存储。这条原则直接决定了片段分类中的排除规则——已经在 System Prompt 中的内容（`AgentsMd`）和临时性的工具调用（`SkillPayload`）都属于「可推导或已存在」的范畴。

真正有价值的是隐式知识——用户的工作习惯、环境偏好、子 Agent 返回的上下文信息。这些信息散落在交互过程中，无法通过简单的文件读取获得，因此被保留在记忆管道中。

1.3 两层记忆的分类

通过片段过滤后的信息，最终进入 Codex 的双层记忆架构：

层次	核心文件	持久性	更新频率	内容性质
静态记忆层	project_doc.rs	磁盘文件 (AGENTS.md)	用户手动编辑	显式声明的规则和偏好
长期记忆层	memories/ 目录	SQLite + 磁盘工件	后台异步管道	自动提取的隐式知识

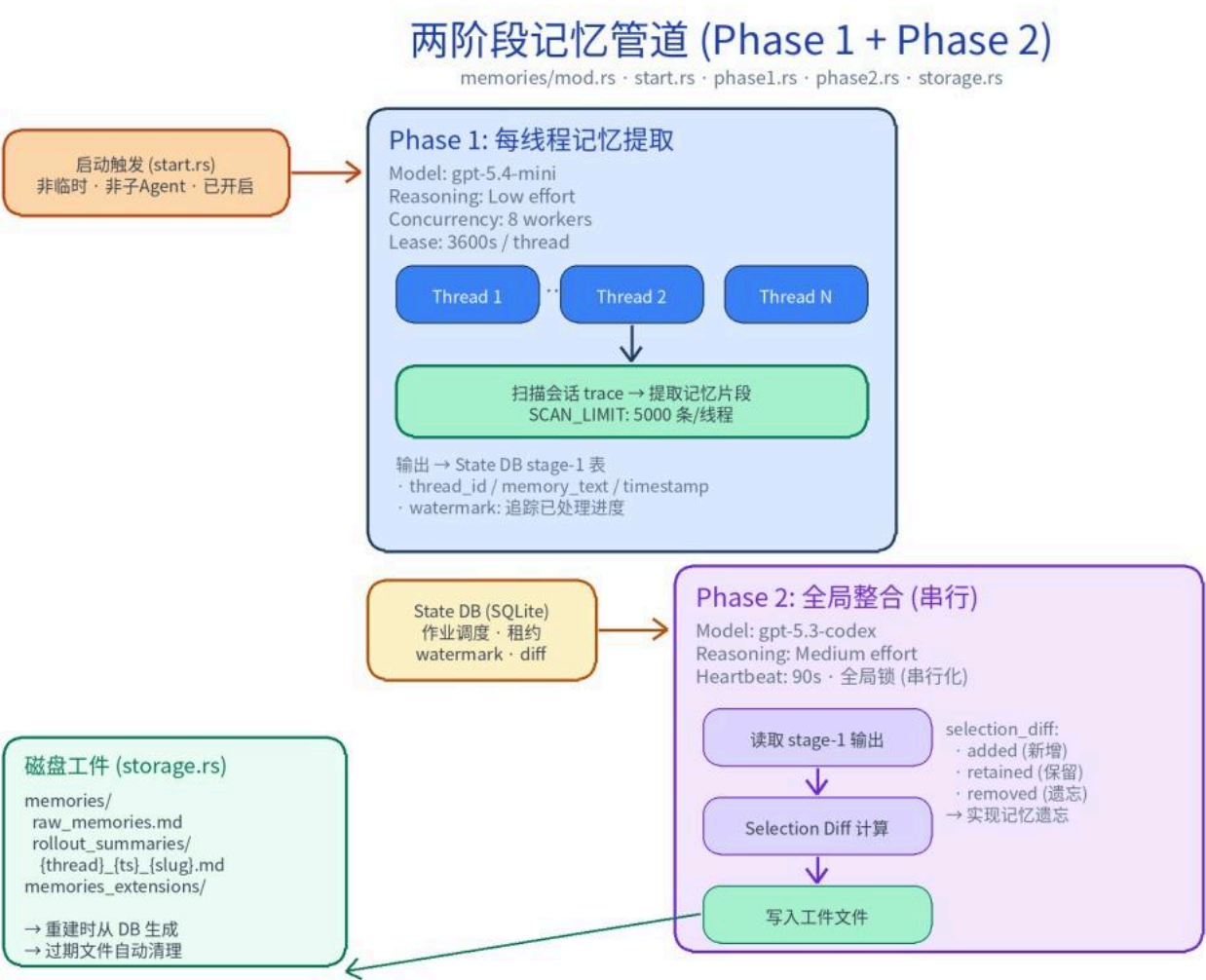
静态记忆层负责「用户告诉 AI 的」，长期记忆层负责「AI 自己观察到的」。两层记忆最终汇聚到 Prompt 构建器，组装成完整的 System Prompt 发送给 LLM。

定义了什么值得记之后，接下来的问题是：谁来执行这个提取过程？何时触发？

二、记忆如何产生——两阶段记忆管道

长期记忆是 Codex 记忆系统中最复杂的部分。它通过一个精心设计的两阶段管道，从历史交互中自动提取有价值的知识并整合为持久化记忆。

2.1 管道架构



整个管道由 `memories/start.rs` 启动，必须满足以下前置条件：

代码块

```
1 // codex-rs/core/src/memories/start.rs
2 pub(crate) fn start_memories_startup_task(
3     session: &Arc, config: Arc, source: &SessionSource,
4 ) {
5     if config.ephemeral // 非临时会话
6         || !config.features.enabled(Ftecture::MemoryTool) // 功能已开启
7         || matches!(source, SessionSource::SubAgent(_)) // 非子 Agent
8     { return; }
9     if session.services.state_db.is_none() { return; } // State DB 可用
10
11     tokio::spawn(async move {
12         phase1::prune(&session, &config).await; // 清理过期 stage-1 输出
13         phase1::run(&session, &config).await; // Phase 1: 并行提取
14         phase2::run(&session, config).await; // Phase 2: 全局整合
15     });
16 }
```

五层门控确保管道只在合适的条件下启动——临时会话、子 Agent、功能未启用等情况下都不会触发，避免不必要的资源消耗。

2.2 Phase 1：水平扩展的线程级提取

Phase 1 的设计目标是水平扩展——同时处理多个会话线程，从每个线程中提取有价值的记忆片段。

配置项	值	设计意图
模型	gpt-5.4-mini	轻量模型，降低成本
推理强度	Low	提取不需要深度推理
并发上限	8	最多同时处理 8 个线程
租约时长	3600 秒	每个线程作业的最长处理时间
线程扫描上限	5000 个线程	限制每次启动扫描的候选线程数

Phase 1 通过 State DB 进行作业调度，流程如下：

- 租约声明：Worker 从 State DB 声明一个线程作业，获取租约
- 消息扫描：加载该线程的 rollout 内容

- 记忆提取：调用 gpt-5.4-mini 提取有价值的记忆片段
- 结果写入：将提取的记忆写入 State DB 的 stage-1 表
- Watermark 更新：记录已处理到的位置，支持增量处理

Watermark 机制借鉴了[消息队列](#)（如 Kafka consumer offset）的设计——每次运行只处理上次 watermark 之后的新消息，避免重复处理。这使得 Phase 1 可以高效地增量运行，而不需要每次都从头扫描全部历史。

2.3 Phase 2：全局串行的整合与遗忘

Phase 2 是一个全局串行的过程——同一时刻只有一个 Phase 2 任务在运行。这是因为它需要对所有 Phase 1 的输出进行整合，并会导致不一致。

配置项	值	设计意图
模型	gpt-5.4	全尺寸模型，具备更强的归纳整合能力
推理强度	Medium	整合需要理解和归纳
心跳间隔	90 秒	保持作业活跃的心跳
并发	全局独占声明（串行）	确保整合的一致性

Phase 2 的核心创新是 Selection Diff 机制——它不仅整合新记忆，还对已有记忆进行重新评估：

代码块

```
1 Phase2InputSelection:
2   selected:
3     • [added]      → 上次 Phase 2 后新产生的 stage-1 输出
4     • [retained] → 上次 Phase 2 已选且仍保留的 stage-1 输出
5   removed:
6     • 上次 Phase 2 选中但本次不再保留的 stage-1 输出（触发遗忘）
```

这实现了[记忆遗忘](#)——AI 不只是无限积累记忆，而是能够判断哪些记忆已经过时或不再相关，主动移除它们。没有遗忘能力的记忆系统会逐渐被噪声淹没，这是一个至关重要的设计。

2.4 Memory Trace：另一条输入路径

除了两阶段管道，`memory_trace.rs` 提供了另一条记忆输入路径——从 trace 文件加载记忆：

代码块

```
1 pub async fn build_memories_from_trace_files(
2     client: &ModelClient,
```



```
3    trace_paths: &[PathBuf],
4    model_info: &ModelInfo,
5    effort: Option,
6    session_telemetry: &SessionTelemetry,
7 ) -> Result> {
8    // 1. 逐个 trace 文件: 加载文本 → 解析 JSON/JSONL → 构建 API 请求体
9    let mut prepared = Vec::with_capacity(trace_paths.len());
10   for (index, path) in trace_paths.iter().enumerate() {
11       prepared.push(prepare_trace(index + 1, path).await?);
12   }
13   // 2. 批量调用模型生成摘要
14   let output = client.summarize_memories(raw_memories, model_info, effort,
15   ...).await?;
16   // 3. 组装 BuiltMemory { memory_id, source_path, raw_memory, memory_summary
17   }
18   Ok(prepared.into_iter().zip(output).map(|(trace, summary)| BuiltMemory {
19   ... }).collect())
20 }
```

该模块支持 BOM 检测和容错解码，能够处理各种格式的 trace 文件（JSON 数组和 JSONL），为记忆系统提供了除实时对话之外的批量导入能力。

2.5 两阶段模式的经济学

Phase 1 用 gpt-5.4-mini（便宜、快），Phase 2 用 gpt-5.4（贵、强）。这种模型分工反映了一个实用的经济学考量：

维度	Phase 1	Phase 2
并发模型	多 Worker 并行 (≤8)	全局单一任务
模型选择	小模型 (mini)	大模型 (codex)
推理强度	Low	Medium
数据范围	单线程局部	跨线程聚合
调度方式	租约 + 重试	全局独占声明
核心职责	批量初步提取	高价值整合与遗忘决策

不是所有任务都需要最强的模型。**批量提取用小模型完成，整合决策用大模型完成——这种「粗筛 + 精炼」的模式在 AI 应用架构中越来越常见。**

三、记忆存在哪里——双层存储架构

记忆产生之后，需要一个可靠的存储层来承载它们。**Codex 采用双层存储架构，分别应对不同的使用场景。**

3.1 静态记忆层：AGENTS.md 的层级发现

静态记忆是最直观的存储形式——**用户直接编写规则文件，告诉 AI 该做什么、不该做什么。Codex 选了一个优雅的方案：AGENTS.md 层级发现。**

AGENTS.md 层级发现与合并机制

project_doc.rs · discover_project_doc_paths() · read_project_docs_with_fs()

文件系统层级

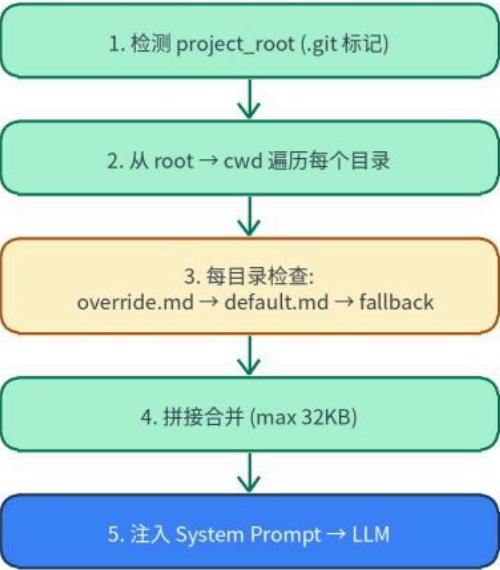


优先级递增 ↓

关键约束:

- project_doc_max_bytes: 32KB
- project_root_markers: [.git]
- fallback: 可自定义文件名
- 覆盖时整个目录层替换
- 空文件跳过

发现流程 (discover_project_doc_paths)



核心代码逻辑

```
for dir in root..=cwd {
  if override.exists() → use override
  elif default.exists() → use default
  elif fallback.exists() → use fallback
}
```

知乎 @Meta

project_doc.rs 定义了两个关键常量：

代码块

```
1 pub const DEFAULT_PROJECT_DOC_FILENAME: &str = "AGENTS.md";
2 pub const LOCAL_PROJECT_DOC_FILENAME: &str = "AGENTS.override.md";
```

AGENTS.md 文件可以存在于多个层级，从全局到局部形成覆盖链：

层级	路径示例	典型内容	作用域
全局	~/.codex/AGENTS.md	个人偏好：编码风格、语言习惯	所有项目共享
项目根	/project/AGENTS.md	团队规范：架构约定、测试策略	单个项目
子目录	/project/src/api/AGENTS.md	模块规则：特定API的约束	特定目录
覆盖	任意目录AGENTS.override.md	临时覆盖：紧急修改、实验性规则	替代同级AGENTS.md

发现算法的核心逻辑：

代码块

```
1 // 简化后的核心逻辑（实际实现还包含 fallback filenames 和异步 fs 调用）
2 async fn discover_project_doc_paths(config: &Config, fs: &dyn
  ExecutorFileSystem)
3     -> Vec
4 {
5     // 1. 从 cwd 向上查找 project_root_markers（默认 .git）确定项目根
6     // 2. 收集从项目根到 cwd 路径上的所有目录
7     // 3. 每个目录按优先级检查候选文件：AGENTS.override.md > AGENTS.md > fallbacks
8     let candidate_filenames = [LOCAL_PROJECT_DOC_FILENAME,
  DEFAULT_PROJECT_DOC_FILENAME, ...];
9     for dir in search_dirs {           // 项目根 → cwd 路径上的每个目录
10         for name in &candidate_filenames {
11             if dir.join(name).is_file() {
12                 found.push(dir.join(name));
13                 break;                 // 同一目录只取第一个匹配
14             }
15         }
16     }
17     found
18 }
```

关键设计决策：

- 项目根检测：通过 `project_root_markers`（默认 `.git`）定位项目根目录
- 合并上限： `project_doc_max_bytes` 默认 32KB，防止注入过多内容占用上下文窗口

- 覆盖优先： `AGENTS.override.md` 存在时替代同级的 `AGENTS.md`，实现临时修改而无需改动团队共享文件
- 空文件跳过：避免空的 `AGENTS.md` 文件干扰合并逻辑

这种层级覆盖的设计借鉴了 CSS 的层叠机制——全局样式可以被局部样式覆盖，`override` 文件提供了不改动原文件的临时覆盖能力。在团队协作中，团队共享 `AGENTS.md`，个人通过 `override` 文件添加自己的偏好，互不干扰。

3.2 长期记忆层：State DB + 磁盘工件

长期记忆的存储由两部分组成：

State DB (SQLite) 是记忆的权威数据源。Phase 1 的提取结果写入 `stage-1` 表，Phase 2 的整合结果写入最终记忆表。State DB 还负责作业调度——通过租约机制管理 Worker 的作业分配和超时重试。

磁盘工件是 State DB 内容的文件系统投影。`storage.rs` 负责将数据库中的记忆持久化为可读的文件：

代码块

```
1  memories/
2    raw_memories.md          # 所有记忆的聚合文件
3    rollout_summaries/      # 每线程摘要文件
4      {thread_uuid}_{timestamp}_{slug}.md
5  memories_extensions/      # 扩展记忆
```

两个关键的同步函数确保磁盘文件始终与数据库状态一致：

- `rebuild_raw_memories_file_from_memories()` — 从 State DB 重建 `raw_memories.md`
- `sync_rollout_summaries_from_memories()` — 写入每线程摘要，清理过期旧文件

3.3 存储层级对比

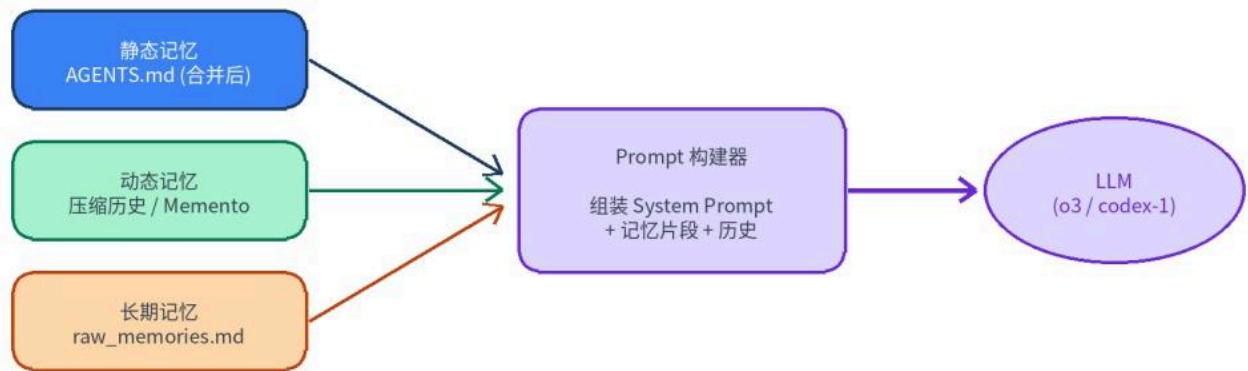
维度	静态记忆层 (AGENTS.md)	长期记忆层 (State DB + 磁盘)
写入者	用户手动编辑	两阶段管道自动写入
持久化方式	纯文本文件	SQLite + Markdown 文件
作用域	全局 / 项目 / 子目录	跨会话全局
更新频率	低频（用户主动修改）	高频（每次会话后异步更新）
大小限制	32KB 合并上限	由 State DB 和磁盘空间决定
覆盖机制	AGENTS.override.md	Selection Diff (added/retained/removed)
在 Prompt 中的位置	System Prompt 用户指令区	System Prompt 长期记忆区

四、记忆如何使用——Prompt 构建与上下文注入

两层记忆最终需要被「消费」——组装成 LLM 可以理解的 Prompt。这个过程涉及精细的过滤和组装逻辑。

记忆消费: Prompt 构建与上下文注入

contextual_user_message.rs · get_user_instructions_with_fs() · Prompt Builder



记忆排除过滤 (contextual_user_message.rs)

排除 (不进入记忆提取)

- AGENTS.md 内容 (已在 System Prompt)
 - Skill payload (临时工具调用)
- is_memory_excluded_contextual_user_fragment()
→ 避免重复和噪声进入记忆管道

保留 (参与记忆提取)

- ENVIRONMENT_CONTEXT (环境上下文)
 - USER_SHELL_COMMAND (用户命令)
 - SUBAGENT_NOTIFICATION (子Agent通知)
 - TURN_ABORTED (中断标记)
- 保留对任务理解有价值的上下文

知乎 @Meta

4.1 组装流程

Prompt 构建器按以下顺序组装最终的 System Prompt:

顺序	内容	来源
①	基础系统指令	硬编码的模型行为指引
②	用户级指令	全局用户配置
③	AGENTS.md 合并内容	project_doc.rs 层级发现结果
④	长期记忆	raw_memories.md 持久化记忆
⑤	对话历史	当前会话的消息序列
⑥	当前输入	用户当前的问题或指令

`get_user_instructions_with_fs()` 负责 ②③ 的合并:

代码块

```
1 pub(crate) async fn get_user_instructions_with_fs(
```

```

2     config: &Config, fs: &dyn ExecutorFileSystem,
3 ) -> Option {
4     let project_docs = read_project_docs_with_fs(config, fs).await;
5     let mut output = String::new();
6     if let Some(instructions) = config.user_instructions.clone() {
7         output.push_str(&instructions);           // 全局用户指令
8         (~/.codex/AGENTS.md)
9     }
10    if let Ok(Some(docs)) = project_docs {
11        output.push_str(&docs);                     // 项目级 AGENTS.md 合并内容
12    }
13    if let Some(js_repl) = render_js_repl_instructions(config) {
14        output.push_str(&js_repl);                 // JS REPL 指令（如果启用）
15    }
16    if config.features.enabled(FEATURE::ChildAgentsMd) {
17        output.push_str(HIERARCHICAL_AGENTS_MESSAGE); // 层级 Agent 消息
18    }
19    if !output.is_empty() { Some(output) } else { None }
20 }

```

4.2 片段过滤的作用

在第一章中提到的片段分类，在此处发挥了关键作用。当对话历史被组装进 Prompt 时，`is_memory_excluded_contextual_user_fragment()` 决定了哪些片段会被标记为「记忆排除」——这些片段虽然参与当前对话的 Prompt 构建，但不会被后台的两阶段管道拾取进行记忆提取。

这形成了一个精巧的闭环：

- 静态记忆（AGENTS.md）通过 System Prompt 注入 → 其对应的片段被标记为 `memory_excluded` → 不会再被长期记忆管道重复提取
- 长期记忆通过 `raw_memories.md` 注入 → 在对话中发挥作用 → 新的对话内容有可能触发下一轮的记忆提取

4.3 防御性措施

Prompt 构建过程中嵌入了多项防御性措施：

- 大小上限：AGENTS.md 合并后不超过 32KB，防止占用过多上下文窗口
- 历史裁剪：对话历史超出上下文窗口时，逐条移除最早的消息直到 Prompt 适合模型窗口
- 权限保护：敏感数据文件使用 `0o600` 权限，仅所有者可访问
- 文件锁 + 重试：`message_history.rs` 使用 10 次重试、100ms 间隔的文件锁策略，防止并发读写冲突

五、记忆如何维护——遗忘、增量更新与清理

记忆系统的长期健康运行，依赖于有效的维护机制。Codex 在这方面的设计同样精细。

5.1 Selection Diff：有选择的遗忘

Phase 2 的 Selection Diff 机制是 Codex 记忆维护的核心。每次 Phase 2 运行时，模型不仅整合新提取的记忆，还会对已有的全部记忆进行重新评估：

分类	含义	后续动作
added	新发现的、有价值的记忆	写入 State DB 和磁盘
retained	已有且仍然相关的记忆	保持不变
removed	不再相关或已过时的记忆	从 State DB 和磁盘中删除

这模拟了人类记忆的特性——不再相关的信息会被逐渐遗忘，保持记忆库的信噪比。在实际场景中，用户的偏好会变化，项目的技术栈会演进，过时的记忆不仅无用，还会干扰当前决策。

5.2 Watermark：增量处理的保障

Watermark 机制确保 Phase 1 的增量高效运行。每个线程维护一个处理位置的标记（watermark），下次运行时只处理该标记之后的新消息。

这种设计带来两个好处：

- 效率：不需要每次都从头扫描全部历史，大幅减少计算开销
- 幂等性：即使 Worker 意外中断后重启，从 watermark 处继续即可，不会遗漏也不会重复

5.3 过期文件清理

`sync_rollout_summaries_from_memories()` 在同步每线程摘要文件时，还会清理已经不再需要的旧文件。当某个线程的记忆在 Phase 2 中被标记为 removed 后，对应的摘要文件也会从磁盘上删除，防止存储空间无限增长。

5.4 租约超时与重试

State DB 的作业调度使用租约机制管理 Worker 的生命周期：

- 租约时长 3600 秒：Worker 必须在此时间内完成作业，否则租约过期，作业重新可用
- 心跳间隔 90 秒：Phase 2 通过心跳保持作业活跃
- 退避重试：失败的作业会以指数退避的策略重新进入队列
- 去重：多个 Worker 同时运行时，租约机制确保同一作业不会被重复处理

六、附：上下文压缩——记忆的邻域机制

值得一提的是，Codex 中还存在一个 [Memento 上下文压缩](#) 机制（`compact.rs`）。虽然它不属于记忆系统，但与记忆的消费密切相关。

压缩解决的是一个不同的问题：当单次会话的对话历史超出上下文窗口时，如何继续工作。它通过调用 LLM 将旧对话生成摘要，替换原始消息，同时保留最近 20K token 的用户消息。这本质上是一种上下文窗口的滑动**管理策略**，不涉及跨会话的知识持久化。

七、设计模式与启示

从源码中，可以提炼出几个值得借鉴的**架构模式**。

7.1 记忆系统的分层思想

Codex 的双层记忆架构给出了一个清晰的分层范式：

- 静态层（AGENTS.md）：显式声明的规则和偏好——高确定性，低频变化，用户完全可控
- 长期层（两阶段管道）：自动提取的隐式知识——低确定性，高价值，系统自主运行

这种分层不仅适用于 AI 编程助手，也适用于任何需要记忆能力的 AI 系统。**关键洞察是：不要试图用一套机制覆盖所有记忆需求——显式规则和隐式知识有着本质不同的生命周期和管理方式。**

7.2 遗忘比记忆更难

记忆系统最容易犯的错误是只增不减。Codex 通过 Selection Diff 实现的遗忘机制是一个重要的设计决策。在实际场景中，过时的记忆不仅无用，还会干扰当前决策。没有遗忘能力的记忆系统，信噪比会持续下降，最终变得不可用。

7.3 「粗筛 + 精炼」的管道模式

Phase 1（水平扩展、小模型、低推理强度）+ Phase 2（全局串行、大模型、中等推理强度）的组合，解决了一个经典的分布式问题：如何在保证最终一致性的同时最大化并发性能。这种模式适用于任何需要从大量原始数据中提炼高质量结果的场景。

7.4 防御性工程实践

源码中随处可见 Rust 系统编程的防御性实践：

实践	具体实现	防护目标
文件锁 + 重试	10 次重试，100ms 间隔	并发读写冲突
权限保护	0o600 文件权限	敏感数据泄露
大小上限	32KB AGENTS.md，逐条 移除最旧消息	资源耗尽
BOM 检测	memory_trace.rs 容错解 码	编码格式兼容
租约超时	3600 秒作业租约	Worker 死锁
Watermark	增量处理位置标记	重复处理和资源浪费

7.5 Rust 的系统编程优势

整个记忆系统用 Rust 实现，充分利用了其所有权系统、类型安全和并发原语。文件锁、异步任务调度、并发控制等在 TypeScript 中需要大量外部库和运行时开销的功能，在 Rust 中通过语言特性和标准库就能优雅实现。这种重写选择本身就体现了 Codex 团队对系统可靠性的追求。

参考资料

- 源码仓库: <https://github.com/openai/codex>
- 核心记忆模块: `codex-rs/core/src/memories/`
- AGENTS.md 发现: `codex-rs/core/src/project_doc.rs`
- 消息历史: `codex-rs/core/src/message_history.rs`
- 记忆追踪: `codex-rs/core/src/memory_trace.rs`
- 上下文分类: `codex-rs/core/src/contextual_user_message.rs`